Punjab University BSCS (CC-311)

Operating Systems Notes by Syeda Laiba

Topic 2: Operating System Services and Structure

1. Operating System Services

Operating systems provide an environment for the execution of programs and offer various services to both users and programs. These services simplify the use of the computer system and ensure efficient resource utilization.

Services for the User

1. **User Interface (UI):** Almost all operating systems provide a user interface. It can be:
2. **Command-Line Interface (CLI):** Text-based interface for typing commands (e.g., Linux terminal, Windows CMD).
3. **Graphical User Interface (GUI):** Visual interface with icons and windows (e.g., Windows, macOS).
4. **Batch Interface:** Commands are collected into files (batch jobs) and executed without user interaction.
5. **Program Execution:** The OS loads programs into memory, runs them, and handles their termination — either normal or abnormal (due to errors).
6. **I/O Operations:** A running program may require I/O (input/output) such as reading from a file or interacting with a device. The OS provides a standard interface for these operations.
7. **File-System Manipulation:** Programs often need to read, write, create, or delete files. The OS manages these tasks and controls permissions to ensure data security.
8. **Communications:** Processes may need to exchange information — either on the same system or across a network. Communication can be via **shared memory** or **message passing**, where packets of data are sent and received by the OS.
9. **Error Detection:** The OS constantly monitors for errors in hardware (CPU, memory), I/O devices, and user programs. It ensures stability by handling errors gracefully and often provides debugging tools for users and programmers.

Services for System Efficiency

1. **Resource Allocation:** When multiple processes or users operate simultaneously, the OS allocates resources like CPU time, memory, and storage efficiently.
2. **Accounting:** Tracks how resources are used to improve performance and for billing in multiuser systems.

3. **Protection and Security:** Prevents unauthorized access and interference between processes. It involves **user authentication** and **access control** to protect data and hardware.

Example: In a multiuser environment, each user logs in with credentials, and the OS ensures that their files are secure from others.

---

2. User Operating System Interface

Users interact with the OS through interfaces designed for ease and functionality.

Command-Line Interface (CLI):

- Allows users to type commands directly.
- Implemented either in the kernel or as a separate shell program.
- Example shells: Bash (Linux), PowerShell (Windows).
- Supports scripting for automation.

Graphical User Interface (GUI):

- User-friendly interface with icons, menus, and windows.
- Uses input devices like mouse and keyboard.
- **Example:** Windows (GUI + CMD), macOS (Aqua GUI + UNIX kernel), and Solaris (CLI + GUI options).

Touch Interface:

- Used in mobile OS (Android, iOS).
- Supports gestures like tap, swipe, pinch.

---

3. System Calls

A **System Call** is the programming interface through which user applications interact with the operating system's kernel services.

They are typically written in high-level languages (like C or C++) and accessed via **Application Programming Interfaces (APIs)** such as: - **Win32 API** (for Windows) - **POSIX API** (for UNIX/Linux/macOS) - **Java API** (for JVM-based programs)

Why APIs Instead of Direct Calls?

APIs provide a consistent and secure way to access system services, hiding complex kernel details from users.

Example: Copying File Using System Calls

To copy one file's contents to another, a program performs system calls like `open()`, `read()`, `write()`, and `close()` sequentially.

System Call Implementation Steps:

1. Each system call has a unique number.
2. The **system call interface** maps the number to the correct kernel function.
3. The kernel executes the service and returns results to the process.
4. Most interactions are managed by **runtime support libraries** like the C Standard Library.

---

4. Types of System Calls

System calls can be categorized as follows:

1. **Process Control:** Manage process creation, termination, and synchronization.
2. Examples: `fork()`, `exec()`, `wait()`, `exit()`
3. **File Management:** Manage files and directories.
4. Examples: `open()`, `read()`, `write()`, `close()`
5. **Device Management:** Control hardware devices.
6. Examples: `ioctl()`, `request()`, `release()`
7. **Information Maintenance:** Manage system data.
8. Examples: `getpid()`, `alarm()`, `time()`
9. **Communication:** Establish and manage connections.
10. Examples: `pipe()`, `send()`, `recv()`
11. **Protection:** Control access rights.
12. Example: `chmod()`, `setuid()`

Example: When running a C program that prints to screen, `printf()` calls the `write()` system call internally.

---

5. System Programs

System programs provide a convenient environment for program development and execution. They act as user-level interfaces to system calls.

Types of System Programs:

- **File Management:** Create, delete, copy, or rename files.
- **Status Information:** Provide system info (date, time, memory usage).
- **File Modification:** Editors (e.g., Notepad, vim) allow editing.
- **Programming Support:** Compilers, linkers, debuggers.
- **Program Loading and Execution:** Loaders and debuggers manage execution.
- **Communication Programs:** Allow file transfer, email, or chat.

Example: The UNIX `bash` shell, compilers like GCC, and file management utilities are all system programs.

---

6. Operating System Design and Implementation

Designing and implementing an OS is complex and depends on system goals, hardware, and intended use.

Goals:

- **User Goals:** Easy to use, safe, fast.
- **System Goals:** Efficient, flexible, and maintainable.

Key Principle:

- **Policy:** Defines *what* will be done.
- **Mechanism:** Defines *how* it will be done.
- Example: Policy — choose which process to run; Mechanism — implement CPU scheduler.

Example: UNIX was designed in C for portability, separating policy and mechanism for flexibility.

---

7. Operating System Structure

OS structures determine how components interact and how responsibilities are divided.

Common Structures:

1. **Simple Structure:** (e.g., MS-DOS) — minimal modularity, all functions in one layer.
2. **Layered Approach:** OS divided into layers, each relying on lower ones. Example: THE OS, Windows NT.
3. **Microkernel:** Moves core services (like I/O, communication) to user space. Example: macOS, QNX.
4. **Modules:** Uses loadable kernel modules; object-oriented and flexible (used in Linux).
5. **Hybrid:** Combines microkernel and monolithic features (Windows, macOS).

---

8. Virtual Machines (VMs)

Virtual Machines treat hardware and the OS kernel as software, allowing multiple OSs to run concurrently on the same hardware.

Benefits:

- Isolation and protection between systems.
- Easier testing and development.
- Efficient hardware utilization.

Examples:

- VMware, VirtualBox, Hyper-V.

Real-World Use:

Developers use VMs to test software across different OS environments.

History and Features:

- Originated with IBM mainframes (1972).
- Each VM behaves as a complete computer system.
- Can share files and communicate securely.

---

9. Operating System Generation

Each OS must be configured for specific hardware — a process called **OS Generation**.

Steps:

1. Hardware configuration via **SYSGEN program**.
2. Selection and linking of required modules.
3. Creation of initialization files for the boot process.

Example: UNIX uses configuration files to include device drivers and system parameters during setup.

---

10. System Boot

The **boot process** initializes the system and loads the operating system into memory.

Boot Steps:

1. **Power-On Self-Test (POST):** Checks hardware.
2. **Bootstrap Loader:** Stored in ROM, locates and loads the OS kernel.
3. **Kernel Initialization:** Starts system services.
4. **System Ready:** The OS becomes ready for user commands.

Example: On startup, BIOS/UEFI runs POST, loads GRUB, and then boots Linux or Windows.

Quick Revision Cards

- **OS Services:** UI, execution, I/O, file management, communication, error detection.
- **Efficiency Services:** Resource allocation, accounting, protection.
- **User Interface:** CLI, GUI, Touch.
- **System Calls:** Interface between programs and kernel.
- **Types:** Process, File, Device, Info, Communication, Protection.
- **System Programs:** Tools for development and management.
- **Design:** Goals, Policy vs. Mechanism.

- **Structures:** Simple, Layered, Microkernel, Modular, Hybrid.
- **Virtual Machines:** Multiple OS on one machine.
- **OS Generation:** Hardware-specific configuration.
- **System Boot:** Sequence from power-on to OS ready.

Prepared by: *Syeda Laiba*
Course: BSCS (5th Semester), Punjab University A.C.P